

RISC-V Vectorization Coverage for HPC: A TSVC-Based Analysis

Hung-Ming Lai
National Tsing Hua University
Hsinchu, Taiwan
Lawrence Livermore National
Laboratory
Livermore, USA
hmlai@pplab.cs.nthu.edu.tw

Pei-Hung Lin
Lawrence Livermore National
Laboratory (LLNL)
Livermore, USA
lin32@llnl.gov

Maya Gokhale
Lawrence Livermore National
Laboratory
Livermore, USA
gokhale2@llnl.gov

Ivy Peng
KTH Royal Institute of Technology
Stockholm, Sweden
ivybopeng@kth.se

Hiren Patel
University of Waterloo
Waterloo, Canada
hdpatel@uwaterloo.ca

Jenq-Kuen Lee
National Tsing Hua University
Hsinchu, Taiwan
jklee@cs.nthu.edu.tw

Abstract

The RISC-V Vector Extension (RVV) introduces scalable, vector-length agnostic operations with strong potential for high-performance computing (HPC). This paper presents a TSVC-based instruction coverage analysis of RVV to evaluate current compiler auto-vectorization support. We compile TSVC with GNU and LLVM under both vector-length agnostic (VLA) and vector-length specific (VLS) modes and analyze the emitted instructions against the RVV 1.0 specification. Our results quantify instruction usage across key groups, identify missed instructions, and classify the causes of failed vectorization, including compiler backend limitations, absent use cases in TSVC, and nontrivial or unsupported patterns. We also highlight TSVC's limitations, including ambiguous kernel vectorizability and missing representations of modern HPC-relevant patterns. Finally, we suggest directions for enhancing benchmark suites to better reflect RVV capabilities and guide compiler development for HPC workloads.

CCS Concepts

• Software and its engineering → Compilers; Extra-functional properties; • Computer systems organization → Parallel architectures.

Keywords

RISC-V, Compiler, Vectorization, TSVC, Benchmark

ACM Reference Format:

Hung-Ming Lai, Pei-Hung Lin, Maya Gokhale, Ivy Peng, Hiren Patel, and Jenq-Kuen Lee. 2025. RISC-V Vectorization Coverage for HPC: A TSVC-Based Analysis. In *Workshops of the International Conference for High Performance*

Computing, Networking, Storage and Analysis (SC Workshops '25), November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 8 pages.
<https://doi.org/10.1145/3731599.3767535>

1 Introduction

The RISC-V instruction set architecture (ISA) [3] has emerged as an open and extensible alternative to proprietary processor architectures, attracting significant interest across research, industry, and high-performance computing (HPC). A key component of RISC-V for computationally demanding workloads is the RISC-V Vector Extension (RVV), which provides scalable, vector-length-agnostic operations suitable for a wide range of HPC applications. Achieving performance on modern architectures, however, depends heavily on compiler auto-vectorization: the ability of compilers to automatically transform loops and kernels into vectorized forms that exploit available hardware instructions.

Benchmark suites play a central role in evaluating and guiding compiler vectorization. The Test Suite for Vectorizing Compilers (TSVC) and its derived versions [6, 12, 18] have long been used to study auto-vectorization effectiveness, offering a diverse collection of loop kernels designed to reveal compiler strengths and weaknesses [2, 4, 12, 15]. It has also been collected into the LLVM compiler test suite. However, TSVC was originally designed with legacy vector ISAs in mind (e.g., SSE and preceding instruction sets) and does not fully capture newer architectural features such as those provided by RVV. Moreover, TSVC suffers from a key limitation: the ground truth of whether a kernel is truly vectorizable is not always well-defined. Some kernels exhibit subtle data dependencies or control-flow patterns where vectorization is only possible if the compiler's analysis and transformation passes can establish certain assumptions (e.g., relaxed numerical accuracy). The extent to which these assumptions are recognized and leveraged varies by compiler, making vectorization contingent on the level of analysis support and optimization strategy. This dependency complicates both automated reasoning about legality and the interpretation of observed compiler behavior.

In this work, we conduct a comprehensive instruction coverage analysis of RVV 1.0 using TSVC-2, an updated version of TSVC with several issues fixed. We compile the benchmark suite with GNU GCC and LLVM Clang compilers, under both vector-length

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SC Workshops '25, St Louis, MO, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1871-7/25/11

<https://doi.org/10.1145/3731599.3767535>

agnostic (VLA) and vector-length specific (VLS) modes, and analyze the emitted assembly against the official RVV specification. This allows us to quantify which RVV instructions are utilized, which are entirely absent, and the degree of coverage achieved in different instruction groups.

Beyond statistics, we also examine the causes of missed instructions and failed vectorization cases, classifying them into three categories: (1) compiler backend or pattern-matching limitations, (2) lack of representative use cases in TSVC, and (3) nontrivial vectorization requirements or unsupported cases. This root-cause analysis provides insight into both compiler maturity and benchmark limitations.

While this study does not extend TSVC directly, we highlight future directions for benchmark enhancement to better represent RVV capabilities and support compiler development. In particular, we suggest incorporating kernels for masked operations, segmented loads/stores, mixed-precision arithmetic, and permutation patterns, features with clear relevance to HPC workloads such as stencils, sparse linear algebra, and numerical solvers.

The contributions of this paper are as follows.

- A TSVC-based coverage study of RVV instruction usage across GNU and LLVM, under VLA and VLS modes.
- Analysis of missed instructions and failed vectorization cases with classification of underlying causes.
- Identification of TSVC limitations, including ambiguous vectorizability, and suggestions for extending benchmarks to better represent RVV features relevant to HPC.

The remainder of this paper is organized as follows. Section 2 provides background on RVV and the TSVC benchmark suite. Section 3 describes our methodology for collecting and analyzing instruction coverage. Section 4 presents the observations, including detailed coverage statistics and cause breakdowns for missed instructions. Section 5 discusses the related research work. Finally, Section 6 concludes the paper and highlights future work.

2 Background

2.1 RISC-V & RVV

RISC-V is an open-standard ISA based on established Reduced Instruction Set Computer (RISC) principles, designed to be simple, modular, and extensible. Unlike proprietary ISAs, RISC-V is freely available under open licensing, enabling unrestricted implementation for research, education, and commercial use. The base ISA defines a compact set of core instructions, while optional standard extensions (e.g., integer multiplication/division, atomic operations, floating-point arithmetic, vector processing) allow tailoring for diverse application domains, from low-power embedded systems to high-performance computing. Its clean-slate design emphasizes regular instruction encoding, efficient pipelining, and straightforward decoding, facilitating both hardware design simplicity and software toolchain portability. The RISC-V ecosystem has grown rapidly, supported by a global community and a broad range of hardware and software implementations.

The RVV introduces a flexible and scalable vector processing model designed to accelerate data-parallel workloads across a wide range of hardware implementations. Like Arm's Scalable Vector Extension (SVE)[16], RVV adopts a VLA approach, where the same

binary code can execute efficiently on processors with varying vector register lengths. This design enables forward scalability and portability without recompilation, allowing hardware vendors to optimize vector width for performance, area, or power constraints. RVV supports advanced features such as predicated execution, segmented loads and stores, and per-element data type operations, making it well-suited for applications in HPC, machine learning, scientific simulations, and signal processing. Its integration with the RISC-V base ISA ensures a consistent programming model, supported by an evolving compiler toolchain and optimized libraries.

2.2 Auto-vectorization for RVV

Compiler auto-vectorization is a key strategy for exploiting the capabilities of modern vector extensions such as RVV, automatically transforming scalar loops into vectorized code when data dependencies and alignment constraints permit. Advanced compilers employ techniques such as loop vectorization, loop unrolling with vector fusion, and predication to maximize instruction-level parallelism. In the context of vectorization, VLS execution assumes a fixed vector width known at compile time, allowing the compiler to generate straightforward, fully unrolled vector loops. In contrast, VLA execution, as in RVV and Arm SVE, defers the vector length decision to runtime, enabling forward scalability but requiring compilers to emit loops with runtime checks and loop-tail handling that adapt to varying vector widths. While VLS can simplify auto-vectorization and make performance more predictable, VLA provides greater portability across implementations with different register sizes. Due to these differences, compilers for VLA architectures often rely more on advanced loop transformations and runtime code generation, and intrinsics remain important for developers to fully control performance-critical code paths. For RVV, intrinsics are standardized to align with the VLA model, enabling efficient and portable explicit vector programming that complements auto-vectorization [14].

2.3 Benchmark

Performance benchmarks are essential tools for evaluating compiler optimization capabilities, guiding architecture design, and validating the effectiveness of code generation strategies. Among them, the TSVC was specifically created to assess the ability of compilers to detect and exploit data-level parallelism through auto-vectorization. Originally developed in the 1980s by Callahan, Dongarra and Levine [5], TSVC evolved from early vectorization research at Rice University, incorporating a diverse set of small, loop-centric kernels distilled from real-world scientific and engineering codes. Over the years, it has been expanded and modernized to reflect contemporary programming constructs, memory access patterns, and data dependencies that challenge compiler analysis [2, 4, 12, 15]. Today, TSVC is widely used in both academia and industry to measure and compare vectorization effectiveness across architectures, ranging from fixed-width SIMD processors to vector-length-agnostic designs like RVV and Arm SVE, and to identify cases where manual optimization or intrinsics may be needed to achieve peak performance.

3 Methodology

The primary goal of this study is to assess and enhance compiler auto-vectorization support for the RVV in high-performance computing contexts. We adopt a two-phase approach. First, we perform a systematic instruction coverage analysis using the TSVC-2[18] to determine which RVV instructions are currently utilized by major supporting compilers under different vectorization modes. This step establishes a comprehensive view of the existing compiler capabilities and identifies underused or unused instructions.

Second, we use these insights to conduct a deeper investigation into the reasons behind missed instructions and failed vectorization. For each uncovered case, we analyze the generated assembly, compiler diagnostic reports, and relevant code patterns to determine the root cause. These causes can include incomplete compiler backend support, conservative dependence analysis, the absence of representative patterns in TSVC, or nontrivial vectorization requirements that compilers currently avoid. This diagnostic approach allows us to move beyond simple coverage statistics, providing a clearer picture of why certain RVV instructions are not exercised and where gaps remain in current compiler auto-vectorization strategies.

To ensure the relevance of our work to the broader HPC community, we use the RVV-specific analysis to highlight gaps in current benchmark coverage and compiler support. These observations provide guidance for improving benchmark suites such as TSVC, by incorporating kernels that exercise masked operations, segmented memory access, mixed-precision arithmetic, and permutation patterns, so that they better represent modern vector ISAs. Such enhancements would not only capture RVV's unique capabilities but also offer more effective test cases for compiler developers and architecture researchers seeking to evaluate and optimize vectorization strategies for HPC workloads.

3.1 RVV Instruction Coverage Analysis

We first surveyed compiler configurations to identify the most suitable options for RVV auto-vectorization testing. The TSVC was compiled using both GCC v. 15.1.0¹ and Clang v.19.1.7², each in two vectorization modes:

- VLA: the vector length is determined at runtime via `vsetv1` instruction, enabling scalability across implementations with different hardware vector register widths.
- VLS: the vector length is fixed at compile time, allowing the compiler to assume a constant vector width and potentially apply more aggressive unrolling or instruction scheduling.

Table 1 lists the specific compiler options applied for GNU and LLVM for the VLA and VLS modes.

A Python-based parser was developed to process the emitted assembly and match each instruction against the RISC-V Vector Extension specification. Instructions were grouped according to the categories defined in the RVV documentation, including: configuration-setting, load and store, integer, fixed-point, floating-point, reduction, mask and permutations [1]. The analytical result identifies the instruction used and missed by the compiler auto-vectorization and calculates the RVV instruction coverage percentage.

Table 1: Compiler configurations for RVV instruction coverage analysis.

Compiler	Mode	Compiler options	
GNU	VLA	-O3	-ftree-vectorize -march=rv64gcv_zfh_zvfh
GNU	VLS	-O3	-ftree-vectorize -march=rv64gcv_zvl128b -mrvv-vector-bits=zvl
LLVM	VLA	-O3	-fstrict-aliasing -march=rv64gcv -mabi=lp64d -menable-experimental-extensions -mllvm -scalable-vectorization=on
LLVM	VLS	-O3	-fstrict-aliasing -march=rv64gcv -mabi=lp64d -menable-experimental-extensions -mllvm -scalable-vectorization=on -mrvv-vector-bits=128

3.2 Missed Instruction Identification

After completing the RVV instruction coverage analysis, we obtained complete usage statistics for all four compiler/mode configurations (GNU-VLA, GNU-VLS, LLVM-VLA, LLVM-VLS). These statistics provide a full picture of which RVV instructions were emitted, which were entirely absent, and how usage was distributed across the defined instruction groups.

From this global perspective, we identified missed instructions of potential interest for future work. The initial selection step relied purely on coverage data, e.g. any instruction present in the RVV specification but absent from the compiled TSVC output was considered 'missed'. This ensured that the targeting process started from a complete and objective set, without bias from assumptions about compiler capabilities or code structure.

When a missed instruction was flagged, additional investigation could be performed to determine the likely cause:

- Compiler backend or pattern-matching limitations: instruction support not yet implemented or code patterns not recognized for auto-vectorization.
- Lack of representative use cases: TSVC may not contain loops or data access patterns that naturally trigger the instruction.
- Non-trivial vectorization requirements or unsupported cases: instructions needing complex transformations or irregular memory access which compilers may skip without explicit guidance. In some cases, auto-vectorization is not yet supported, even for valid patterns.

This two-step approach, first collecting a complete coverage picture, then investigating root causes, allowed us to distinguish between instructions that are simply absent from the statistics and those that are intrinsically difficult for current compilers to vectorize. This deeper reasoning highlights gaps attributable to compiler limitations versus those arising from the limitations of TSVC itself, and provides insight into where benchmark enhancements could better capture HPC-relevant vectorization patterns in the future.

¹submodule of <https://github.com/riscv-collab/riscv-gnu-toolchain/tree/2025.07.16>

²<https://github.com/llvm/llvm-project/tree/llvmorg-19.1.7>

3.3 Future Benchmark Enhancements

Based on our analysis of missed instructions, we observe that many RVV features are not exercised within the current TSVC suite. For existing benchmarks that show potential to trigger these instructions, one possible direction is to adapt kernels or provide alternative implementations that expose the intended vectorization opportunities. For instance, RVV intrinsic implementations could be developed in future work to demonstrate how the instructions apply in functional kernels, serving both as validation and as reference material for compiler enhancement.

For instructions without a natural match in TSVC, a further opportunity is the design of new microbenchmarks that explicitly target the underrepresented RVV features. These could follow the established TSVC style while incorporating HPC-relevant code patterns, such as masked operations for boundary handling, segmented loads/stores for structured data, or mixed-precision arithmetic common in numerical solvers.

Such extensions would help evolve TSVC, or a successor benchmark suite, to better reflect the capabilities of modern vector ISAs like RVV and SVE, while also providing the compiler and architecture research communities with more representative workloads for evaluating and improving auto-vectorization in HPC contexts.

4 Observation & Discussion

4.1 RVV Coverage Statistics

Table 2 summarizes the RVV 1.0 instruction coverage for GNU and LLVM under both VLA and VLS modes, organized by the instruction groups defined in the RVV 1.0 specification [19]. Coverage percentages are calculated as the number of distinct instructions observed in the generated assembly divided by the total number of instructions in each group. By examining the statistical results alongside the formal instruction definitions and known compiler implementation details, we provide group-wise observations and insights into the patterns of instruction usage and omission.

- **Configuration-setting Instructions:** RVV provides a set of instructions, `vset{i}vl{i}`, to configure the `vl` and `vtype` CSRs. In our TSVC builds, we observe `vsetvli/vsetivli`, but not `vsetvl`. This indicates that the compiler configures `vtype` via the immediate-encoded forms (`vsetvli/vsetivli`) and does not require the register-sourced `vtype` provided by `vsetvl` for these loops. This means scenarios requiring runtime `vtype` selection or save/restore do not arise in TSVC.
- **Load & Store Instructions:** RVV memory operations use the prefixes `vl` for loads and `vs` for stores. Variants include unit-stride (contiguous), strided (`s`), and indexed (`xei`) addressing modes, as well as segmented (`seg`) forms. Indexed instructions additionally have ordered and unordered versions, which determine whether element addresses are accessed sequentially or without ordering guarantees. All memory instructions support masking. Instruction names are grouped by element width, e.g., `vle8`, `vle16`, `vle32`, and `vle64` are all variants of the `vle` (vector load, unit-stride) instruction. The missed instructions in this group for TSVC are primarily load/store mask register (`vlm`, `vsm`), indexed-ordered (`vloxei`, `vsoxei`), strided segmented (`vlsseg`, `vssseg`),

and indexed segmented (`vluxseg`, `vsuxseg`, `vloxseg`, `vsxseg`). TSVC lacks kernels that exercise masked memory operations and advanced segment load/store forms.

For indexed-unordered loads, we observe their presence in TSVC, likely because the compiler's auto-vectorizer can ensure no two vector elements access the same memory location. On the store side, GNU generates indexed-unordered stores, while LLVM emits indexed-ordered stores for memory safety reasons.

We observe that RVV load/store instructions provide significant potential for improving the handling of struct data types in HPC applications. In particular, segmented load/store operations and their variants enable more efficient processing of arrays of structures, which are frequently used to represent complex data layouts in scientific workloads. Additionally, the indexed load/store operations in RVV have strong potential to assist with indirect memory addressing, a common access pattern in large-scale simulation codes. By enabling more efficient handling of irregular memory accesses, which often limit vectorization efficiency, these instructions could broaden the applicability of RVV to real-world HPC kernels.

- **Integer Instructions:** No widening integer instructions, either signed or unsigned, were observed in the TSVC outputs. This absence is largely due to the lack of representative integer-based kernels in the suite. By design, TSVC primarily uses arrays of type `real_t` (floating point only), which leads the compilers to favor floating-point instructions. Nevertheless, modern compilers are capable of generating these integer instructions when suitable integer data types are provided. In the current TSVC outputs, some integer instructions do appear in cases involving type conversions. For example, `vsext` is generated when promoting `uint8_t` values to `uint16_t`, while the reverse conversion makes use of `vnsrl`. These isolated occurrences confirm current compiler support for integer vector instructions, but comprehensive coverage is limited by the floating-point focus of TSVC.
- **Fixed-point Instructions:** None of the RVV fixed-point instructions appear in the compiled TSVC outputs across any compiler or mode. This absence is primarily due to the lack of representative fixed-point computation patterns in the current TSVC benchmark suite. Fixed-point operations, such as saturating arithmetic or fixed-point multiply-accumulate, are not part of TSVC's original design focus, which centers on floating-point workloads rather than integer or fixed-point. As a result, the compiler auto-vectorizers have no opportunity to emit these instructions, highlighting a clear gap in benchmark coverage for this category. The code snippets in Listing 1 demonstrates the current compiler's capability for generating fixed-point instructions.

```
void vaadd_example(int32_t *restrict dst,
                  const int32_t *restrict a,
                  const int32_t *restrict b,
                  size_t n)
{
    for (size_t i = 0; i < n; ++i) {
        dst[i] = (a[i] + b[i]) >> 1;
    }
    // vaadd.vv is generated by LLVM v.19
```

Instruction group	config-setting	Load & Store	Integer	Fixed-Point	Floating-Point	Reduction	Mask	Permutation	Sum of all groups
RVV Instruction count	3	21	55	13	41	16	15	15	179
GNU-VLA	2 66%	10 47.6%	9 16.4%	0 0%	10 24.4%	1 6.3%	1 6.7%	8 53.3%	41 22.9%
GNU-VLS	2 66%	11 52.4%	9 16.4%	0 0%	12 29.3%	1 6.3%	1 6.7%	8 53.3%	44 24.6%
LLVM-VLA	2 66%	10 47.6%	10 18.2%	0 0%	17 41.4%	3 18.8%	1 6.7%	9 60%	52 29.1%
LLVM-VLS	2 66%	10 47.6%	10 18.2%	0 0%	17 41.4%	3 18.8%	1 6.7%	8 53.3%	51 28.5%

Table 2: RVV 1.0 instruction count and coverage percentage in TSVC

}

Listing 1: Example of fixed-point, vaadd, instruction

- **Floating-point Instructions:** Similar to integer instructions, no widening floating-point instructions were observed in the TSVC outputs, primarily due to the absence of representative kernels that exercise such operations. Similarly, most FMA-style (fused multiply-add) instructions are missing. TSVC does not naturally expose computation patterns that trigger FMAs, even though modern compilers are fully capable of generating them under typical optimization settings (Listing 2). Several other floating-point math instructions are also absent, reflecting the lack of representative workloads in TSVC. In some cases, enabling fast-math compiler options can generate these vector instructions, e.g., `vfsqrt`, but this often trades accuracy for performance by allowing approximations and less precise results. Nonetheless, we confirmed that compilers such as LLVM v.19 are able to emit some of the missing instructions when applied to suitable code patterns. A few examples are presented below to demonstrate this capability, although overall TSVC coverage for floating-point instructions remains incomplete.

```

a[i] = (a[i]*b[i]) + c[i]; // FMA: vfmac
a[i] += (a[i]*b[i]);      // FMA: vfmad
a[i] -= (a[i]*b[i]);      // FMA: vfnmsub
a[i] = (b[i]*c[i]) - a[i]; // FMA: vfmsac
ii[i] = (int) a[i];       // Conversion: vfcvt
a[i] = sqrtf(b[i]);       // Math: vfsqrt

```

Listing 2: Example of FMA code patterns

- **Reduction Instructions:** TSVC only partially exercises RVV reduction instructions. LLVM generates `vfredusum` for floating-point sums and also uses `vfredmax`/`vfredmin` for extremum reductions. GCC, by contrast, emits `vfredosum` but does not generate min/max variants. This difference reflects each compiler’s handling of floating-point reduction semantics. Integer and logical reductions (e.g., `vredsum`, `vredand`) are well supported by both compilers and can be triggered with simple loop idioms (Listing 3). However, compiler cannot produce expected widening reductions (e.g., `vwfredsum`), since TSVC kernels do not explicitly promote types. Overall, TSVC covers basic reductions but misses widening forms and provides uneven coverage across compilers.
- **Mask Instructions:** The only RVV mask instruction observed in TSVC-generated code is `vid`, which is often paired with `vrsb` to produce reusable index patterns, as seen in

```

// vfredusum (LLVM), vfredosum (GNU)
sum += a[i];

// vfredmax (LLVM, e.g., s314)
if (a[i] > max_val) {
    max_val = a[i];
}
// vfredmin (LLVM, e.g., s316)
if (a[i] < min_val) {
    min_val = a[i];
}
// Logical reduction: vredand
and_val &= mask[i];

```

Listing 3: Examples of reduction patterns

backward-loop cases such as TSVC kernel `s112`. Vector mask-register logical operations (e.g., `vmand`, `vmor`) are likely beneficial for many loop-based computations, yet none are represented in TSVC. Furthermore, many advanced masking techniques, enabled by the full range of RVV mask instructions, cannot be demonstrated with the current TSVC suite due to the absence of representative patterns.

- **Permutation Instructions:** RVV includes vector slide instructions, which shift vector elements toward higher or lower indices, enabling efficient movement of neighboring values into place for computation. In our TSVC builds, the compilers emit `vsli` and `vsli`, whereas the single-element variants (`vsli`/`vsli`), including their floating-point forms) do not appear, despite their potential usefulness in stencil-like recurrences and related HPC kernels. Integer merge instructions (`vmerge`, `vmv`) are present in TSVC-generated code; however, the floating-point equivalents (`vfmerge`, `vfmv`) are absent, likely due to the lack of kernels that require merging floating-point data. The `vrgatheri16` instruction is also missing from GNU outputs but appears in LLVM outputs, likely reflecting differences in compiler optimization strategies or code generation heuristics.

4.2 VLA vs. VLS

The coverage results in Table 2 indicate only minor differences between VLA and VLS for both the GNU and LLVM compilers. A closer inspection reveals the following distinctions:

- **Load & Store:** For LLVM, coverage is identical between VLA and VLS. With the GNU compiler, however, VLS employs the `vlm` instruction to load a mask in TSVC functions `s252`, `s254`, and `s291`.

- **Floating-Point:** Under GNU, VLS leverages FMA instructions (`vfmadd`, `vfmacc`), whereas VLA relies on `vfmul` and `vfadd` to achieve the same computations. The use of variable vector lengths in VLA likely introduces additional complexity in register management and pipeline scheduling when applying FMA.
- **Permutation:** For LLVM, VLA employs `vrgatheri16` (with 16-bit indices) in TSVC function `s118`, while the VLS variant instead uses `vrgather` with 32-bit indices.

These observations do not fully explain the underlying code generation decisions in either the GNU or LLVM compilers. In essence, the differences largely stem from two factors: the need to balance portability and scalability, and the varying levels of compiler maturity in supporting scalable vectorization for Arm SVE and RISC-V RVV.

4.3 Vector Length (VL) in VLS

VLA vectorization improves portability, since a program can be compiled once and executed across ISAs with different vector lengths. In contrast, VLS vectorization requires recompilation for each target ISA with a different vector length. Table 1 shows the compiler commands for the case of a 128-bit vector length. To evaluate coverage across a broader range, we conduct experiments with vector lengths from 128 to 1024 bits. The vector length is specified using the `-march` option for the GNU compiler and the `-mrvv-vector-bits` option for LLVM.

For the LLVM compiler, varying the vector bit width does not affect coverage results. In contrast, with the GNU compiler, the `vmv.s.x` instruction, which copies a scalar integer register into element 0 of a destination vector register, appears in the generated code once the vector width reaches 512 bits or greater. This instruction occurs only in the TSVC function `s4116`, which computes the sum of an array using indirect addressing to access its elements. Inspection of function `s4116` reveals its relation to the following code:

```
sum += a[off] * aa[j-1][ip[i]];
```

Two distinct code-generation strategies emerge depending on the VL under VLS:

- For $VL < 512$, the running sum resides in a floating-point scalar register. Each chunk seeds the reduction using `vfmv.s.f` with $VL = 1$, and the scalar value is subsequently updated via `vfmv.f.s`, which copies element 0 from a source vector register into a destination scalar floating-point register.
- For wider vectors ($VL \geq 512$), the accumulator remains in vector registers. The reduction is seeded by initializing lane 0 to zero with `vmv.s.x`, followed by a chain of `vfredosum.vs` instructions for floating-point reduction, thereby avoiding scalar floating-point traffic.

This explains why `vmv.s.x` appears only for wide vector lengths in VLS, as the compiler chooses to initialize the vector-resident reduction directly while avoiding both scalar operations and explicit manipulation of the vector length.

4.4 Classification of Auto-Vectorization Gaps

To understand the limits of current compiler auto-vectorization, we examined LLVM's failed vectorization reports on the TSVC benchmark suite. TSVC is specifically designed to include loops that are vectorizable, so the number of failed cases is relatively small - only five distinct failure types were reported. Note that the compiler reports only the first encountered failure, so the listed reason may not reflect the only blocker in a given loop. Nonetheless, the first reported cause still reveals a genuine obstacle to vectorization and provides a foundation for our analysis.

The identified causes of failed vectorization were:

- **Unidentified PHI:** PHI node arising from (1) cyclic data dependencies between instructions, vectorizable only when the pattern is recognized (e.g., induction or reduction), or (2) non-predictable control-flow variables.
- **Memory Conflict:** conservatively assumed aliasing or overlapping memory access patterns.
- **Switch Statement:** presence of a switch construct inhibiting straight-line vector code generation.
- **Non-intrinsic Call:** math library function call with no vectorizable variant.
- **Early Exit:** loops with data-dependent termination (e.g., `break`) preventing vectorization.

Although these failures span diverse causes, they largely fall into two fundamental categories that represent the core barriers to loop vectorization:

Control Flow Complexity. For a loop to be vectorized, each instruction must be executable across multiple iterations in parallel, typically by replacing scalar operations with their vector equivalents (e.g., `add` $\rightarrow$ `vadd`). Constructs such as conditional branches, `switch` statements, and early exits introduce divergent control flow, meaning different iterations may take different execution paths. This divergence prevents straightforward lockstep execution. Modern vector ISAs, such as RVV, address this through masking or predication, enabling selective execution within a vector. Compilers can perform these transformations through if-conversion, but they typically succeed only when the control flow is already simple or has been explicitly rewritten into a predicated form. More complex control flow may require techniques such as speculation [17] to handle.

Data Dependence Ambiguity. Even when all loop operations can be expressed as vector instructions, vectorization must not violate *data dependencies*, especially *loop-carried dependencies*. These include both memory and scalar variable dependencies, such as read-after-write (true dependence) and write-after-read (anti-dependence) across iterations. Failures such as "unidentified PHI" and "memory conflict" reflect cases where the compiler cannot conclusively prove that no harmful dependencies exist, often due to complex SSA structures or insufficient aliasing information. In many TSVC kernels, these conservatively assumed dependencies are not actual semantic barriers, but rather a result of limitations in the compiler's dependence analysis. More complex data dependencies can be addressed through loop interchange to reorder iterations, loop distribution to enable partial vectorization [11, 13], or prefix-sum techniques to handle linear recurrences [10].

4.5 Missed Instruction Analysis in TSVC

RVV’s modern design includes many instructions beyond simple per-lane SIMD operations, enabling vectorization of loop patterns that traditionally pose barriers for compilers, as discussed in the previous section. One such example is shown in Listing 4, taken from the TSVC benchmark. In the original TSVC source, this loop is annotated as “*not vectorizable, the condition cannot be speculated*” due to the presence of an induction variable update inside a conditional branch. This creates a loop-carried dependency on j that conventional auto-vectorizers cannot vectorize without hardware support for per-lane index computation, such as the scan and population-count operations available in RVV.

```
int j = -1;
for (int i = 0; i < (LEN_1D/2); i++) {
    j++;
    a[j] = b[i] + d[i] * e[i];
    if (c[i] > (real_t)0.) {
        j++;
        a[j] = c[i] + d[i] * e[i];
    }
}
```

Listing 4: TSVC loop s123

In scalar form, the value of j in each iteration depends on the number of prior iterations where the condition $c[i] > 0$ held true. This dependency forces sequential execution because later iterations cannot determine their j values until earlier ones are complete. RVV provides operations that can compute these values entirely within a vector iteration, eliminating the scalar dependency chain. In particular:

- `viota.m` performs a masked exclusive prefix sum, producing for each active lane the number of preceding lanes where the mask was true. This gives each lane its own “extra offset” without serial iteration.
- `vcpop.m` counts the number of set bits in the mask for the current vector length, allowing the next chunk’s base j to be advanced in one step.

Together, these instructions effectively implement the inner scalar update logic for j in hardware, preserving semantics while enabling parallel execution, as shown in Listing 5.

To derive the vectorized form in Listing 5 from the scalar code in Listing 4, the compiler would need to go beyond standard auto-vectorization. In particular, it must (i) analyze the loop-carried dependence on j and recognize its equivalence to a prefix-sum of branch outcomes, (ii) convert the conditional into a vector mask (if-conversion), and (iii) transform the computation of per-lane write indices using `viota.m` and `vcpop.m`, and then lower the stores to masked indexed vector stores (`vsuxei.m`). Together, these analyses allow the scalar recurrence to be expressed in parallel form.

While both Clang and GCC already perform if-conversion, they lack idiom recognition for the masked prefix-sum recurrence on j and do not lower it to `viota.m/vcqp.m`, as the required legality checks are not yet implemented. This is not a fundamental shortcoming of current compiler analysis capabilities, but rather a reflection that RVV is new and exposes advanced features not previously available, making it a natural direction for future compiler work.

```
1 // original j starts at -1, then ++j before first store
2 j_base = 0
3 // processed elements so far
4 p = 0
5
6 while (p < N/2) {
7     vl = vsetvl(N/2 - p)
8
9     // Load streams
10    vB = vle(b + p, vl)
11    vC = vle(c + p, vl)
12    vD = vle(d + p, vl)
13    vE = vle(e + p, vl)
14
15    // Common computation: t = d[i] * e[i]
16    vT = vmul(vD, vE)
17    vBval = vadd(vB, vT) // b[i] + d[i]*e[i]
18    vCval = vadd(vC, vT) // c[i] + d[i]*e[i]
19
20    // Mask for branch condition: m[i] = (c[i] > 0)
21    m = vgt(vC, 0)
22
23    // Exclusive prefix popcount of m
24    vIO = viota.m(m)
25
26    // Indices for first store: a[j] = b[i] ...
27    vI = vid()
28    idx_b = j_base + vI + vIO
29    vsuxei(a, idx_b, vBval)
30
31    // Indices for second store: a[j] = c[i] ... (masked)
32    idx_c = idx_b + 1
33    vsuxei.m(a, idx_c, vCval, m)
34
35    // Advance j_base for next chunk
36    extra = vcqp.m(m)
37    j_base += vl + extra
38    p += vl
39 }
```

Listing 5: Vectorized pseudocode for TSVC s123 using advanced mask instructions in RVV

5 Related Work

Previous studies have assessed compiler auto-vectorization across multiple ISAs, revealing both shared challenges and architecture-specific considerations. The evaluation by Brank, [7] identifies limitations in vector-length agnostic loop recognition, dependency resolution, and mask handling—issues that are conceptually relevant to the RVV due to their similar flexible vector-length model. In contrast, Comparison of Vectorization Capabilities of Different Compilers for X86 and ARM CPUs [15] adopts a cross-ISA perspective, showing that compiler performance varies not only by architecture but also by toolchain, with x86 AVX-512, Arm NEON, and SVE benefiting differently from GCC, Clang/LLVM, and ICC heuristics and cost models. While these works offer insight into mature SIMD ISAs, A Performance Analysis of Autovectorization on RVV RISC-V Boards [7] provides the first systematic look at RVV in practice, showing incomplete coverage of vectorizable patterns, less efficient handling of vector-length agnostic code, and measurable performance gaps compared to hand-tuned RVV intrinsics. However, this prior RVV-focused study does not explore the impact of new instructions introduced in the evolving RVV specification, such as advanced segment load/store operations, widening/narrowing arithmetic, and masked reduction instructions. The absence of

such evaluations leaves a gap in understanding how current compilers exploit these emerging features, especially in comparison to the mature instruction sets of SVE and AVX-512, and motivates targeted research into optimization strategies and compiler code generation for these RVV-specific capabilities.

6 Conclusion

This study presented a TSVC-based analysis of instruction coverage for the RVV 1.0 under multiple compilers and vectorization modes. Our results show that while basic vector operations such as configuration-setting, contiguous loads/stores, and simple arithmetic are well covered, large gaps remain in advanced categories including reductions, permutations, masked operations, and mixed-width arithmetic. Many of these gaps arise not only from compiler limitations but also from the absence of representative kernels within TSVC itself. In addition, we identified ambiguity in TSVC regarding the ground truth of vectorizability for some kernels, which complicates both evaluation and interpretation of compiler behavior.

Although the focus of this paper has been on observations of missed instructions and failed vectorization cases, our findings highlight the need for a reworking of TSVC. The original design, while effective for earlier vector ISAs such as SSE and preceding instruction sets, does not sufficiently address the capabilities of modern vector processors like SVE and RVV. A carefully extended suite would improve representativeness for HPC workloads, incorporate kernels that exercise advanced instruction classes, and provide clearer criteria for vectorizability.

Furthermore, several specific issues in TSVC warrant improvement. First, benchmarks should more clearly define the ground truth of whether a kernel is vectorizable, removing ambiguity caused by data dependencies, alignment assumptions, or compiler-specific heuristics. Second, benchmarks should support a broader range of input data sizes, enabling evaluation of compiler behavior across small-loop, cache-bound, and large-scale memory-intensive cases. Finally, modern HPC usage patterns should be reflected, such as kernels inspired by RAJA[9] or Kokkos[8], which rely heavily on modern C++ abstractions and expose different challenges for compilers targeting RVV or SVE.

Such a reworked benchmark suite would not only benefit the compiler community, by exposing optimization opportunities and guiding backend development, but also support the architecture research community, by clarifying how emerging vector ISAs can be applied to real HPC applications. In this way, benchmarks can play a central role in ensuring that the evolution of compilers and architectures proceeds hand-in-hand with the demands of high-performance scientific computing.

Looking forward, a redesigned benchmark could also serve as a systematic assessment framework for compiler vectorization capability. By capturing coverage across instruction classes and mapping them to performance outcomes, it would provide a practical metric for comparing both compiler maturity and vector architecture design choices. This, in turn, would enable more meaningful performance assessments of compilers and architectures alike, creating a stronger foundation for co-design of future HPC systems.

Acknowledgments

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344. LLNL-CONF-2011150.

References

- [1] 2022. RISC-V Vector Extension Web Page (RVV documentation). <https://rvv-isadoc.readthedocs.io/en/latest/index.html#>. Revision available online; built with Sphinx and Read the Docs.
- [2] Neil Ait and Adrian Sampson. 2022. Performance left on the table: An evaluation of compiler autovectorization for RISC-V. *IEEE Micro* 42, 5 (2022), 41–48.
- [3] Krste Asanović and David A Patterson. 2014. Instruction sets should be free: The case for risc-v. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146* (2014).
- [4] Bine Brank and Dirk Pleiter. 2022. Assessing the State of Autovectorization Support based on SVE. In *2022 IEEE International Conference on Cluster Computing (CLUSTER)*. 556–562. doi:10.1109/CLUSTER51413.2022.00073
- [5] D. Callahan, J. Dongarra, and D. Levine. 1988. Vectorizing compilers: a test suite and results. In *Supercomputing '88: Proceedings of the 1988 ACM/IEEE Conference on Supercomputing, Vol. I*. 98–105. doi:10.1109/SUPERC.1988.44642
- [6] David Callahan, Jack J Dongarra, and David Levine. 1988. *Vectorizing compilers: A test suite and results*. Argonne National Laboratory.
- [7] Lorenzo Carpentieri, Mohammad VazirPanah, and Biagio Cosenza. 2025. A Performance Analysis of Autovectorization on RVV RISC-V Boards. In *2025 33rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP)*. 129–136. doi:10.1109/PDP66500.2025.00026
- [8] H Carter Edwards, Christian R Trott, and Daniel Sunderland. 2014. Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. *Journal of parallel and distributed computing* 74, 12 (2014), 3202–3216.
- [9] Richard D Hornung and Jeffrey A Keasler. 2014. The RAJA portability layer: overview and status. (2014).
- [10] Hung-Ming Lai, Jenq-Kuen Lee, and Yuan-Shin Hwang. 2023. Enhancing LLVM Optimizations for Linear Recurrence Programs on RVV. In *Proceedings of the 52nd International Conference on Parallel Processing Workshops* (Salt Lake City, UT, USA) (*ICPP Workshops '23*). Association for Computing Machinery, New York, NY, USA, 79–87. doi:10.1145/3605731.3605904
- [11] Hung-Ming Lai, Yan-Rong Pan, and Jenq-Kuen Lee. 2025. Devise Loop Distribution with Scalar Expansion for Enhancing Auto-Vectorization in LLVM. In *2025 EuroLLVM Developers' Meeting*. Berlin, Germany. https://llvm.org/devmtg/2025-04/slides/technical_talk/lai_loop_distribution.pdf
- [12] Saeed Maleki, Yaoqing Gao, Maria J. Garzarán, Tommy Wong, and David A. Padua. 2011. An Evaluation of Vectorizing Compilers. In *2011 International Conference on Parallel Architectures and Compilation Techniques*. 372–382. doi:10.1109/PACT.2011.68
- [13] Sjoerd Meijer, Madhur Amilkwanthar, and Sebastian Pop. 2024. Loop Vectorisation: a quantitative approach to identify/evaluate opportunities. In *2024 LLVM Developers' Meeting*. Santa Clara, CA, USA. <https://llvm.org/devmtg/2024-10/slides/techtalk/Meijer-Loop-Vectorisation.pdf>
- [14] RISC-V Vector C Intrinsic Task Group. 2025. RISC-V Vector C Intrinsic Specification. <https://github.com/riscv-non-isa/rvv-intrinsic-doc/releases/tag/v1.0-ratified>. Version v1.0, Ratified.
- [15] Nazmus Sakib, Tarun Prabhu, Nandakishore Santhi, John Shalf, and Abdel-Hameed A. Badawy. 2024. Comparison of Vectorization Capabilities of Different Compilers for X86 and ARM CPUs. In *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. 1–7. doi:10.1109/HPEC62836.2024.10938481
- [16] Nigel Stephens, Stuart Biles, Matthias Boettcher, Jacob Eapen, Mbou Eyole, Giacomo Gabrielli, Matt Horsnell, Grigorios Magklis, Alejandro Martinez, Nathanael Premillieu, et al. 2017. The ARM scalable vector extension. *IEEE micro* 37, 2 (2017), 26–39.
- [17] Majedul Haque Sujon, R. Clint Whaley, and Qing Yi. 2013. Vectorization past dependent branches through speculation. In *Proceedings of the 22nd International Conference on Parallel Architectures and Compilation Techniques*. 353–362. doi:10.1109/PACT.2013.6618831
- [18] UoB-HPC. 2025. TSVC_2: Updated C version of the Test Suite for Vectorising Compilers. https://github.com/UoB-HPC/TSVC_2. Accessed: 2025-08-16.
- [19] Andrew Waterman and Krste Asanović (eds.). 2025. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V International. <https://riscv.org/technical/specifications/>. Document Version 20250508.